A classification of well-behaved graph clustering schemes

Vilhelm Agdur

28 September 2023

1 Introduction

general stuff about clustering and why it is an interesting problem

some talk about why optimization-based methods have poor theoretical guarantees

some talk about why an axiomatic approach gives algorithms that run in polynomial time, unlike the generally NP-hard optimization problems

soft heuristic statement of what our axioms/properties are and why they might be desirable

statement of our results

2 Definitions of our terms

Definition 1. The category of simple graphs $\mathcal{G}$ has as its objects simple graphs, which we consider to be a finite set V equipped with a reflexive and symmetric binary relation E – that is, our simple graphs are finite, have no double edges, and have loops at every vertex[1]. A morphism from $H = (V, E)$ to $G = (V', E')$ is an injective set function $f : V \to V'$ such that whenever uEv, we also have $f(u)E'f(v)$. That is, a morphism from H to G is a way of realizing H as a subgraph of G.

Definition 2. The category of partitioned sets $\mathcal{P}$ has as its objects sets V with an equivalence relation $\sim$. A morphism in $\mathcal{P}$ from $(V, \sim)$ to $(W, \approx)$ is a set function $f : V \to W$ such that whenever $v \sim w$, $f(v) \approx f(w)$. So in the case where $V = W$, $f = \mathrm{Id}$ is a morphism if $\approx$ is a coarsening of $\sim$.

Definition 3. A clustering scheme is a function[2] $\mathfrak{C}$ that sends simple graphs to partitioned sets, with the same underlying set. We say that it is *functorial* if it is a functor from $\mathcal{G}$ to $\mathcal{P}$.

Definition 4. A clustering scheme $\mathfrak{C}$ is *excisive* if, for all graphs $G = (V, E)$, it holds for every equivalence class A of $\mathfrak{C}(G)$ that $\mathfrak{C}(G|_A) = (A, A^2)$. That is, if we take a graph, cluster it, pick out one of the parts, and cluster that part alone, that part will be clustered into a single part.

[1]The final property here may seem a bit unusual, but it simplifies our proofs in some places, without really affecting much.

[2]Technically, of course, neither $\mathcal{G}$ nor $\mathcal{P}$ are small categories, so the word "function" should be read as "class function" here, not in the usual sense of the word. We sweep this issue under the rug here, because it will never affect anything.

3 Representability

Definition 5. Given a not necessarily finite set Ω of simple graphs, we define the endofunctor F_Ω on $\mathcal{G}$ as follows: For any graph $G = (V, E)$, $F_\Omega(G)$ is a graph on the same vertex set, and there is an edge between u and v in $F_\Omega(G)$ whenever there exists some subgraph of G containing both u and v which is isomorphic to some $\omega \in \Omega$.

Equivalently, there is an edge uv in $F_\Omega(G)$ whenever there exists an $\omega \in \Omega$ and a morphism $f : \omega \to G$ such that $f(\omega) \ni u, v$.

On morphisms, F_Ω is always just the identity.

Lemma 6. *For any representing set Ω of simple graphs, F_Ω is indeed an endofunctor on $\mathcal{G}$.*

Proof. That it always gives a simple graph and that it respects identity morphisms and composition of morphisms is easy to see. The only thing we actually need to check is that if $f : H \to G$ is a morphism, then $F_\Omega(f) = f : F_\Omega(H) \to F_\Omega(G)$ is also a morphism.

Unwrapping this statement, what we need is that, for any morphism $f : H \to G$, whenever uv is an edge in $F_\Omega(H)$, $f(u)f(v)$ is an edge in $F_\Omega(G)$. Now, that uv is an edge in $F_\Omega(H)$ means that there is an $\omega \in \Omega$ and $f_\omega : \omega \to H$ such that $f(\omega) \ni u, v$.

But consider what happens if we just compose f_ω and f – this will be a morphism from ω into G, and trivially we must have that $f(f_\omega(\omega)) \ni f(u), f(v)$, and so there must be an edge $f(u)f(v)$ in $F_\Omega(G)$, by definition of F_Ω. □

What is going on in the proof of Lemma 6 is much clearer if we just think about it in terms of things being subgraphs of each other, forgetting the data of *how* exactly they are subgraphs. Then the statement that F_Ω is a functor boils down to the statement that if ω is a subgraph of H and H is a subgraph of G, then certainly ω is a subgraph of G, and so $F_\Omega(G)$ must have all the edges $F_\Omega(H)$ has, since edges in $F_\Omega(G)$ represent the presence of a certain subgraph in G.

Definition 7. The functor $\Pi : \mathcal{G} \to \mathcal{P}$ which sends a graph to its partition into connected components is called the connected components functor. Equivalently, we can think of it as taking the equivalence relation hull of the binary relation E defining the edges.

Definition 8. A clustering scheme $\mathfrak{C}$ is *representable* if we can write it as $F_\Omega \circ \Pi$ for some representable endofunctor F_Ω. Notice that this means that all representable clustering schemes are trivially functorial.

One somewhat surprising fact is that there are actually representable clustering schemes that are not finitely representable.

Theorem 9. *There exists a representable endofunctor F_Ω which is not* finitely *representable, that is, it is not equal to F_Γ for any finite set of simple graphs Γ.*

In order to give a proof of this, we first need a structural result about how these endofunctors behave.

Remark 10. For any $\omega \in \Omega$, it is trivial to see that $F_\Omega(\omega)$ must be a clique, since we can just take the identity morphism on ω to show that all edges must exist.

In fact, which graphs are sent to cliques by F_Ω essentially determines what it does as a functor. We will see this even more precisely in the next section, but for now this lemma will suffice for our needs:

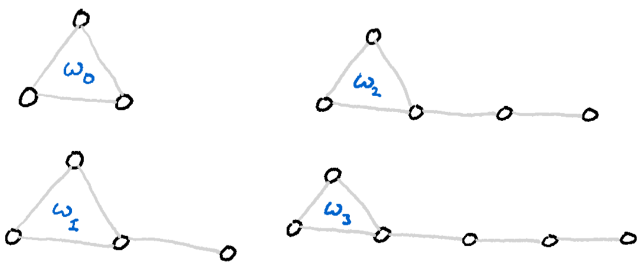

Figure 1: The family of graphs $\{\omega_i\}_{i=0}^3$.

Lemma 11. *For any set Ω of simple graphs and any simple graph $G = (V, E)$, it holds that $F_{\Omega\cup\{G\}} = F_\Omega$ if and only if $F_\Omega(G) = (V, V^2)$, that is, if G is sent to a clique on its vertices by F_Ω.*

Proof. Suppose $F_{\Omega\cup\{G\}} = F_\Omega$. That $F_{\Omega\cup\{G\}}(G) = (V, V^2)$ is immediate by Remark 10, and so since $F_{\Omega\cup\{G\}} = F_\Omega$, we also have $F_\Omega(G) = (V, V^2)$, as desired.

Now suppose $F_\Omega(G) = (V, V^2)$, and let H be some arbitrary simple graph. We wish to show that $F_{\Omega\cup\{G\}}(H) = F_\Omega(H)$.

That edges of $F_\Omega(H)$ must also be edges of $F_{\Omega\cup\{G\}}(H)$ is obvious, so all we need to show is that edges of $F_{\Omega\cup\{G\}}(H)$ are also edges of $F_\Omega(H)$. So suppose uv is an edge of $F_{\Omega\cup\{G\}}(H)$ – that this is so must be because there is some $\omega \in \Omega \cup \{G\}$ and an $f : \omega \to H$ such that $f(\omega) \ni u, v$.

If this ω is not G, then we are done. If it is G, we recall that since $F_\Omega(G)$ is a clique, there must be an edge $f^{-1}(u)f^{-1}(v)$ in it. Therefore there must exist some $\omega \in \Omega$ and some $g : \omega \to G$ such that $f(\omega) \ni f^{-1}(u), f^{-1}(v)$. But then the composition $g \circ f : \omega \to H$ will be a morphism from an $\omega \in \Omega$ into H whose image contains u and v, proving that uv is an edge of $F_\Omega(H)$. □

With this result in hand, we can now give a proof of the existence of a representable but not finitely representable endofunctor.

Proof of Theorem 9. Let, for each i, ω_i consist of a triangle with a tail of i vertices, as shown in Figure 3, and let $\Omega = \{\omega_i \mid i \in \mathbb{N}\}$. We claim that F_Ω is not equal to F_Γ for any finite set of simple graphs Γ.

To prove this, let us first define

$$\bar{\Omega} = \left\{G \in \mathcal{G} \;\middle|\; F_\Omega(G) = (V_G, V_G^2)\right\}$$

as the set of all things which F_Ω sends to cliques. By Lemma 11, $F_\Omega = F_{\bar{\Omega}}$, and $\bar{\Omega}$ is the greatest set with this property.

A little bit of reflection will make it clear that $\bar{\Omega}$ is precisely the set of connected graphs which contain a triangle. Particularly, for any edge in such a graph, we can map the tail end of some ω_i onto that edge, the rest of the tail onto a path to the triangle in the graph, and the triangle of the ω_i onto the triangle of the graph. That F_Ω will send any triangle-free graph to a graph with no edges is clear, since there is no morphism from any triangle-containing graph such as ω_i into a triangle-free graph. Likewise, that it cannot send a disconnected graph to a clique is obvious.

Now suppose for contradiction that Γ is a finite set of graphs such that $F_\Gamma = F_{\bar{\Omega}}$. Lemma 11 implies that we must have $\Gamma \subset \bar{\Omega}$, since if there were a $\gamma \in \Gamma \setminus \bar{\Omega}$ it'd be sent to a clique by F_Γ, and thus also by $F_{\bar{\Omega}}$, since we've assumed they are equal – but we chose $\bar{\Omega}$ so that it already contains everything sent to a clique by $F_{\bar{\Omega}}$.

So, let

$$\delta = \max_{\gamma \in \Gamma} \max \{d(u, v) \mid u, v \in V_\gamma, v \text{ is in a triangle}\}.$$

That is, δ is the furthest any vertex in any graph in Γ is from a triangle. We claim that F_Γ will not send $\omega_{\delta+1}$ to a clique, proving it is not equal to $F_{\bar{\Omega}}$, giving our contradiction.

That this is so is actually easy to see – if $\omega_{\delta+1}$ were sent to a clique, that must in particular mean there is an edge in $F_\Gamma(\omega_{\delta+1})$ between the tail vertex and its neighbour. So there is some morphism from some $\gamma \in \Gamma$ that hits both vertices. However, such a morphism must also map the triangle in γ onto the triangle in $\omega_{\delta+1}$, since that is the only place the triangle can go. Then, however, it cannot also reach to the tail – the tail is further away from the triangle than any vertex in any γ is from the triangle. So no such morphism can exist, and we are done. □

Remark 12. The construction in our proof of course works equally as well replacing the triangle by any connected graph that is not a line. So we have found a whole lot of examples of classes of graphs that give representable but not finitely representable endofunctors.

The question of which natural classes of graphs are representable or finitely representable may be interesting. The connected graphs, for example, are of course represented by just a K_2.

4 Proof of equivalence of representability and excisiveness

5 Representability for hierarchical clustering